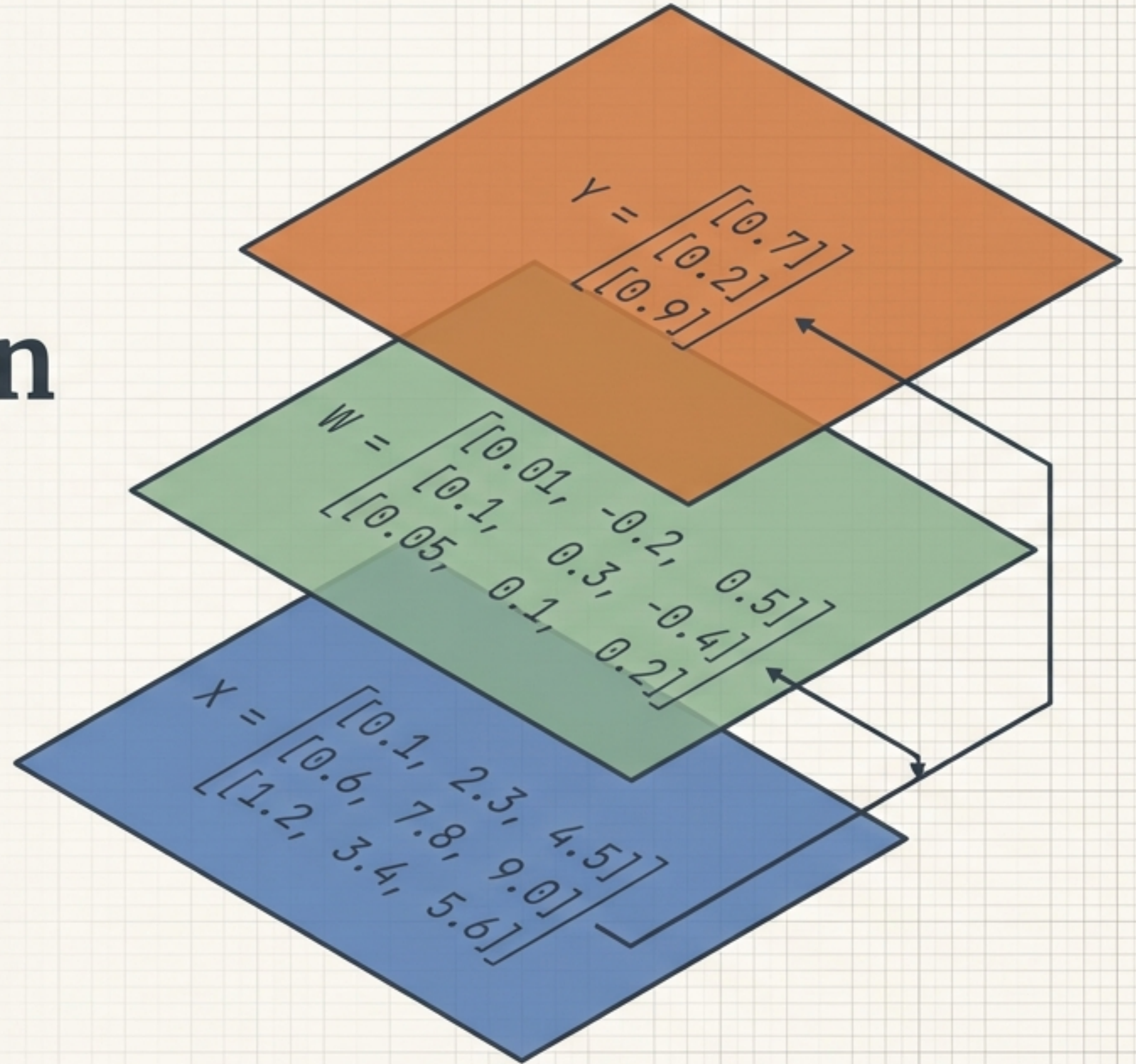


Logistic Regression from Scratch

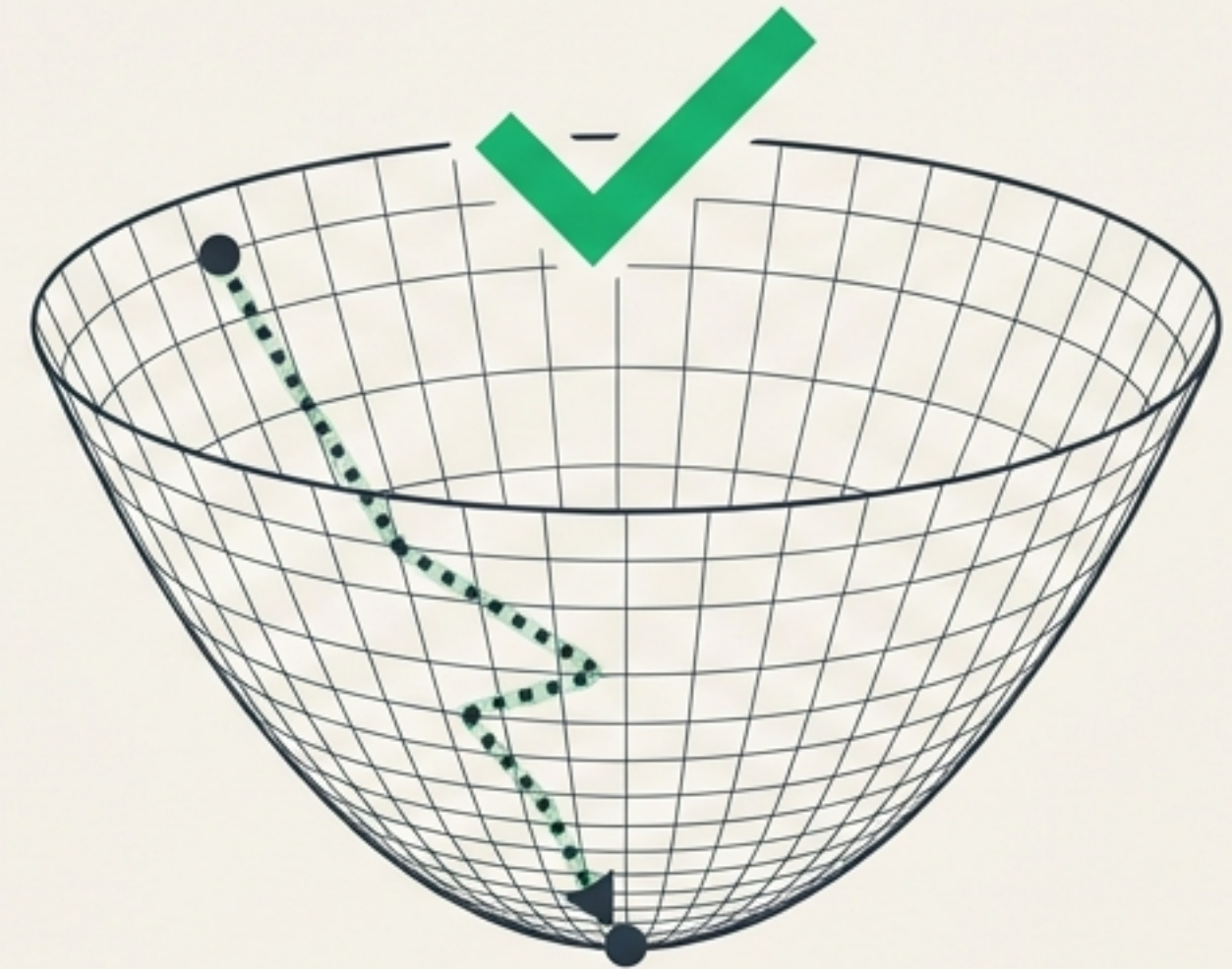
Vectorising Gradient Descent &
Translating Calculus into Python



Why Gradient Descent is non-negotiable

$$\begin{aligned}\Sigma &= \sum_i \sigma_i + \sum_i \sigma_i e^{j\pi} \\ &= \sum_i \sigma_i e^{-j\pi} \\ &= \sum_i \sigma_i W.\end{aligned}$$

The Logistic Regression Loss Function cannot be solved with a simple algebraic formula. There is no closed-form solution.



We must initialise random weights (**W**) and iteratively optimise them using Gradient Descent to find the absolute minimum loss.

Abandoning row-by-row calculations

To write efficient, scalable code, we must compress the entire dataset into a single matrix calculation:

$$Y_hat = \sigma(X \cdot W)$$

Slow For-Loop



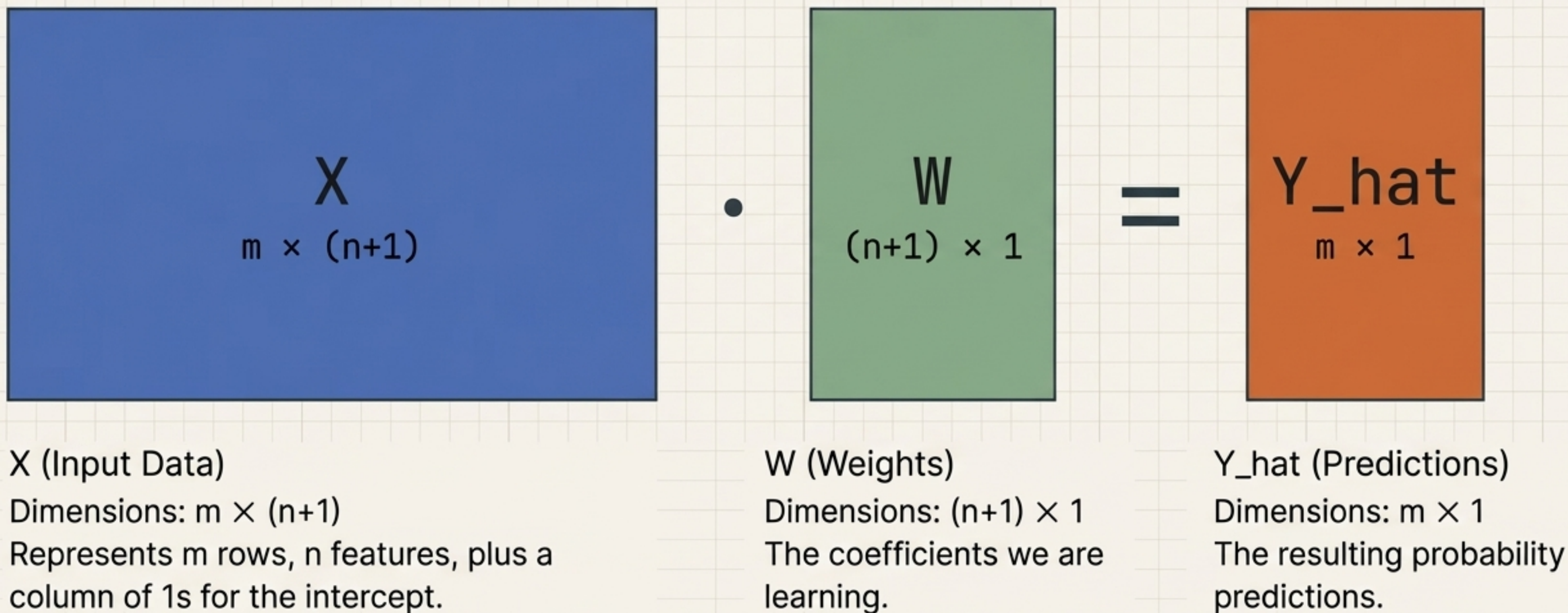
Vectorisation

$$Y_hat = \sigma(X \cdot W)$$

This eliminates iteration overhead and takes advantage of highly optimised linear algebra libraries.

The anatomy of our matrices

Understanding the exact dimensions of these matrices is the secret to vectorised code.
If the dimensions do not align perfectly, the matrix multiplication will fail.



The Loss Function in matrix form

From standard algebra:

$$-(1/m) \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

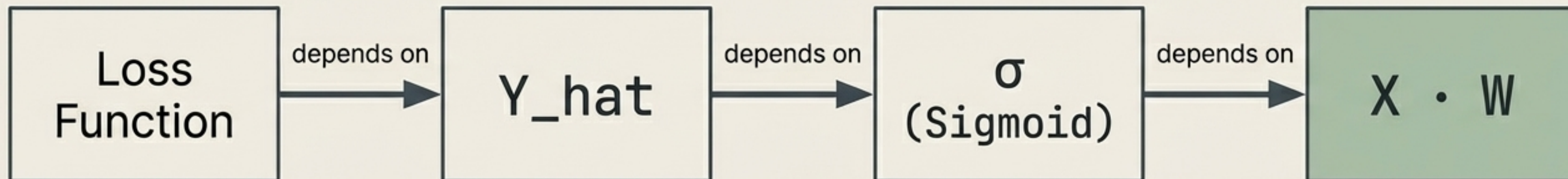
To matrix form:

$$\text{Loss} = -(1/m) [Y \log(\hat{Y}) + (1-Y) \log(1-\hat{Y})]$$

This loop-free formula is the exact target we must differentiate to find our gradient.

Setting up the Chain Rule

To update our weights, we need the derivative of the Loss with respect to W : dL/dW . Because our function is deeply nested, we must systematically apply the Chain Rule to isolate the gradient.



Demystifying the derivative: Term 1

Target: Deriving $Y \log(Y_{\text{hat}})$

Step 1: Derivative of the log.

Step 2: Derivative of the Sigmoid function: $Y_{\text{hat}}(1-Y_{\text{hat}})$

Step 3: Derivative of $X \cdot W$ (which yields X).

Result: Through cancellation, this simplifies beautifully to

$$Y(1-Y_{\text{hat}})X$$

Demystifying the derivative: Term 2

Target: Deriving $(1-Y)\log(1-Y_{\text{hat}})$

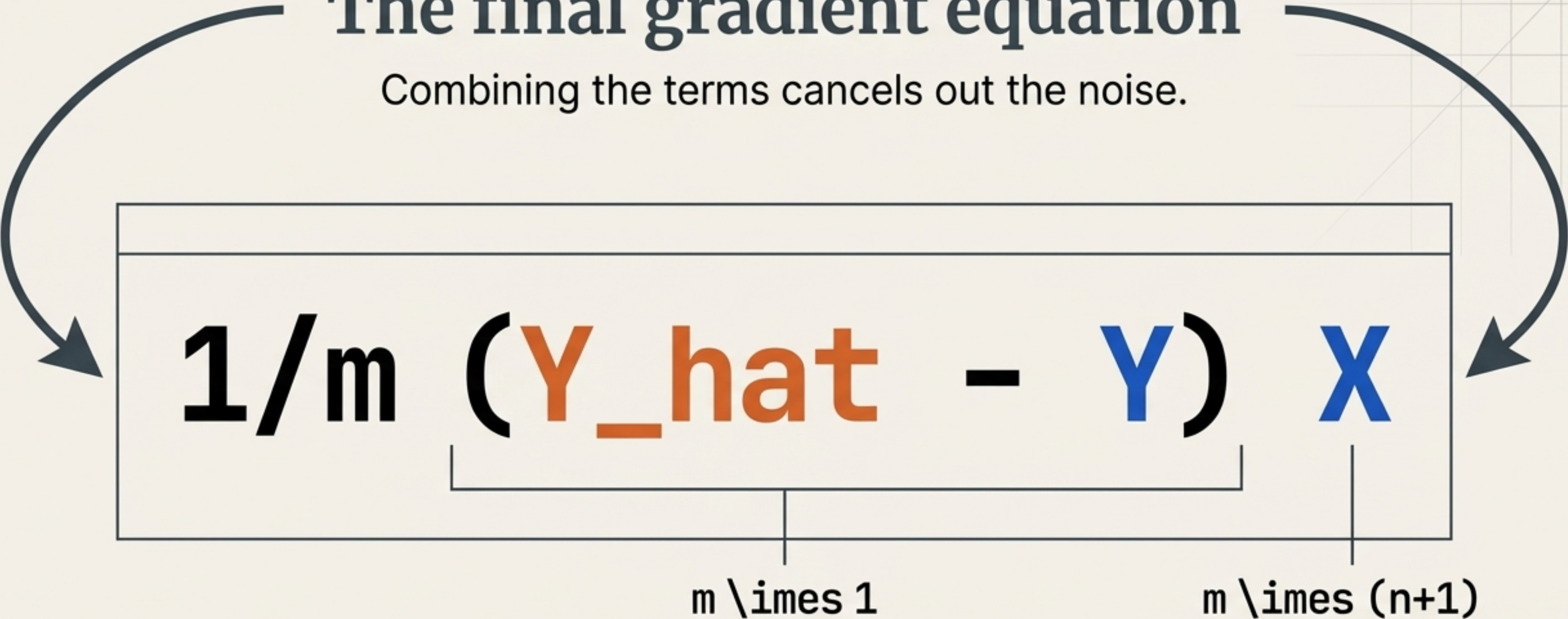
Step 1: Following the exact same Chain Rule logic, but accounting for the negative signs inherent in the $(1-Y)$ structure.

Result: This simplifies to

$$-Y_{\text{hat}}(1-Y)X$$

The final gradient equation

Combining the terms cancels out the noise.



The diagram shows the final gradient equation $\frac{1}{m} (\hat{Y} - Y) X$ enclosed in a large rectangle. Two curved arrows originate from the title 'The final gradient equation' and point to the left and right sides of the rectangle. Below the rectangle, two vertical lines with horizontal bars at the bottom indicate dimensions. The first line is positioned under the term $(\hat{Y} - Y)$ and is labeled $m \times 1$. The second line is positioned under the term X and is labeled $m \times (n+1)$.

$$\frac{1}{m} (\hat{Y} - Y) X$$

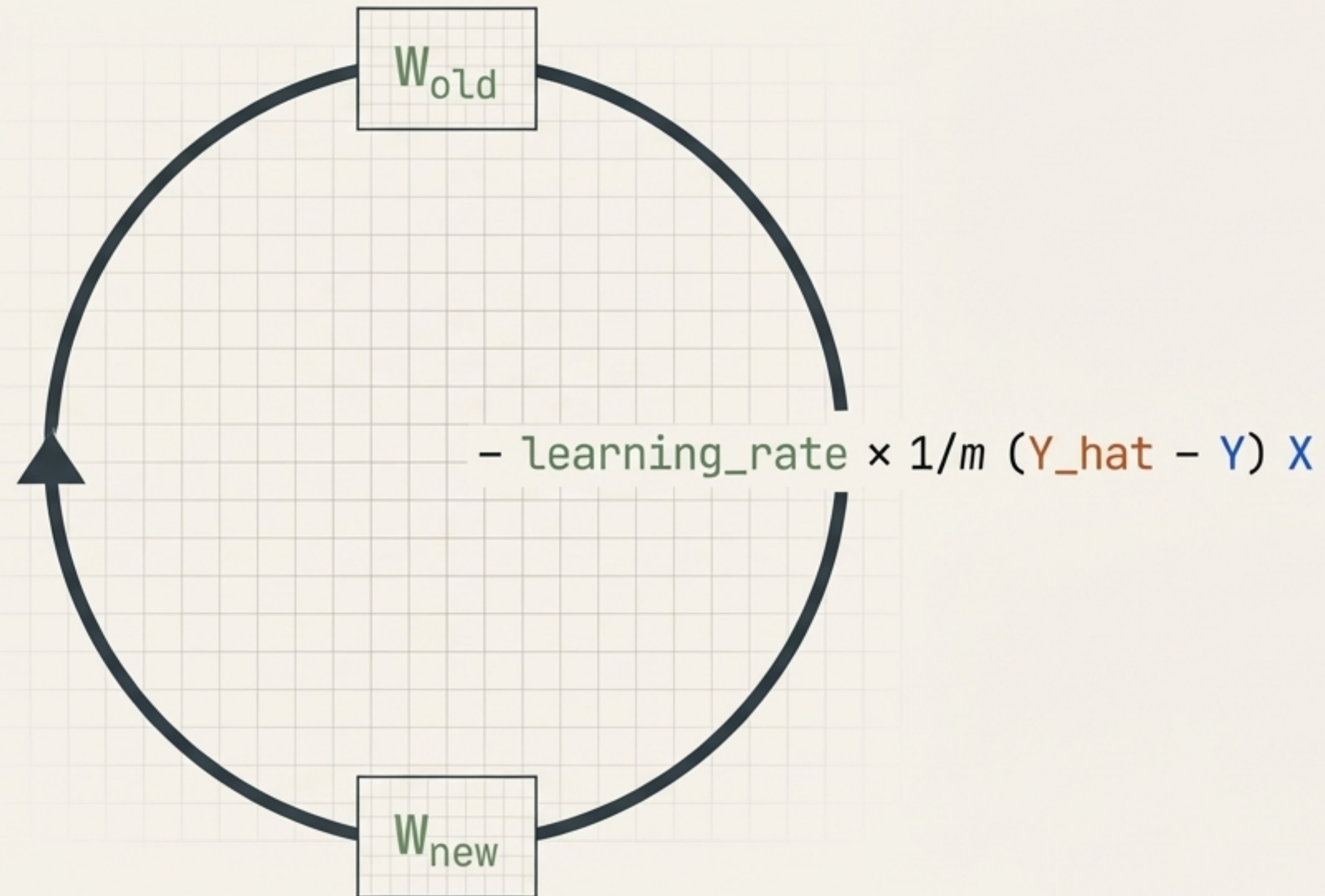
$m \times 1$

$m \times (n+1)$

Dimensionality Check: $m \times 1$ interacting with $m \times (n+1)$ perfectly preserves the dimensions needed to update the $(n+1) \times 1$ weight matrix.

The Gradient Descent update rule

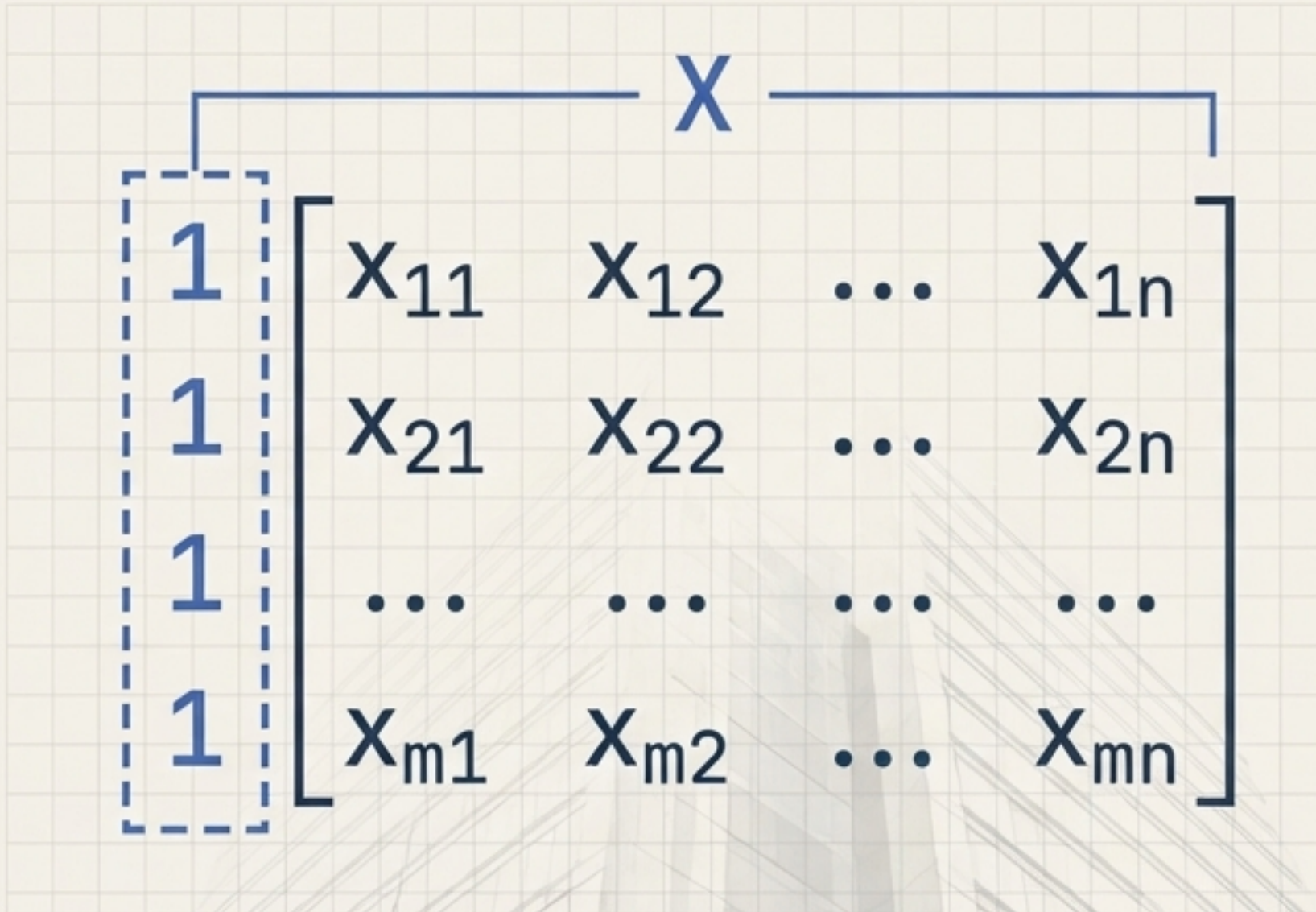
The Learning Loop: $W_{\text{new}} = W_{\text{old}} - \text{learning_rate} \times 1/m (Y_{\text{hat}} - Y) X$



Loop this equation over N epochs. With every pass, the weights adjust slightly, settling only when the overall loss is minimised.

Preparing the input data in Python

The maths requires a W_0 intercept, so X needs a column of 1s. This single line of numpy seamlessly appends the intercept column to our dataset matrix before we initialise the weights.



```
np.insert(X, 0, 1, axis=1)
```

Translating the mathematical loop into code

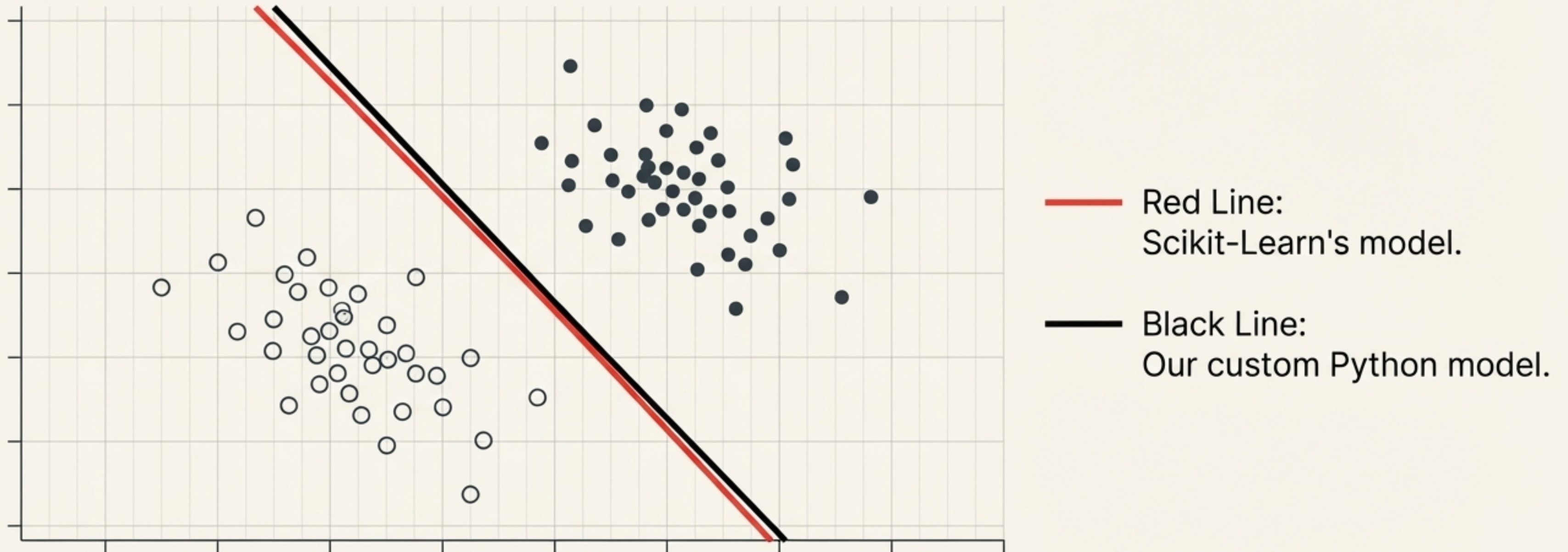
```
y_hat = sigmoid(np.dot(X, weights))  
weights -= learning_rate * (np.dot((y_hat - y), X) / m)
```

$$\mathbf{Y_hat} = \sigma(\mathbf{X} \cdot \mathbf{W})$$

$$\mathbf{W} = \mathbf{W} - lr \times \frac{1}{m} (\mathbf{Y_hat} - \mathbf{Y}) \mathbf{X}$$

The moment of truth: Custom vs. Scikit-Learn

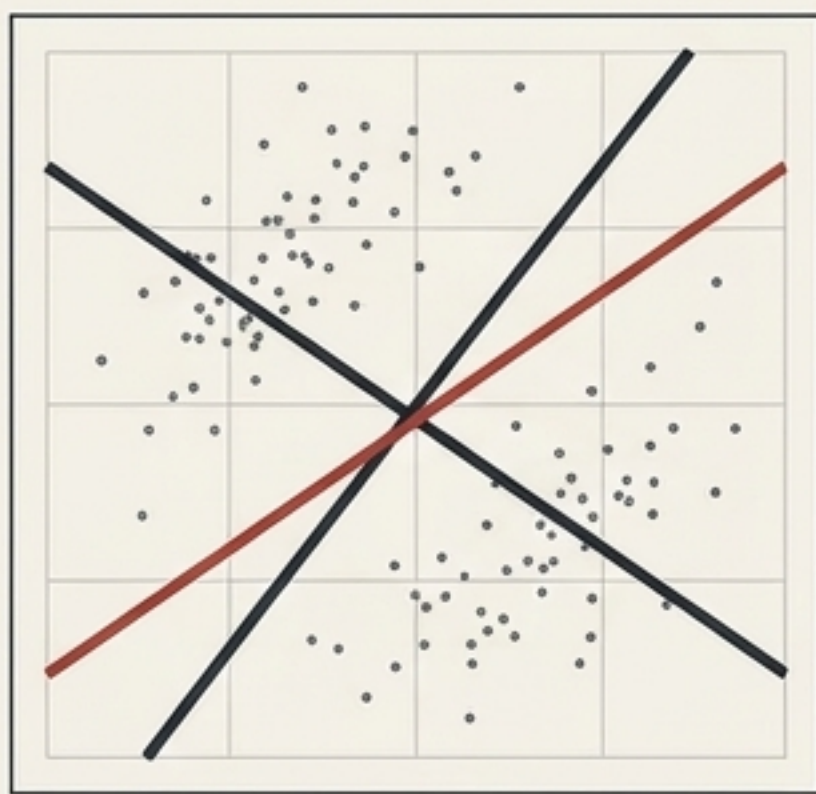
Running the exact same dataset through Scikit-Learn's `LogisticRegression(solver='sag')` and our custom Gradient Descent function. The decision boundaries are nearly identical. The maths holds up in practice.



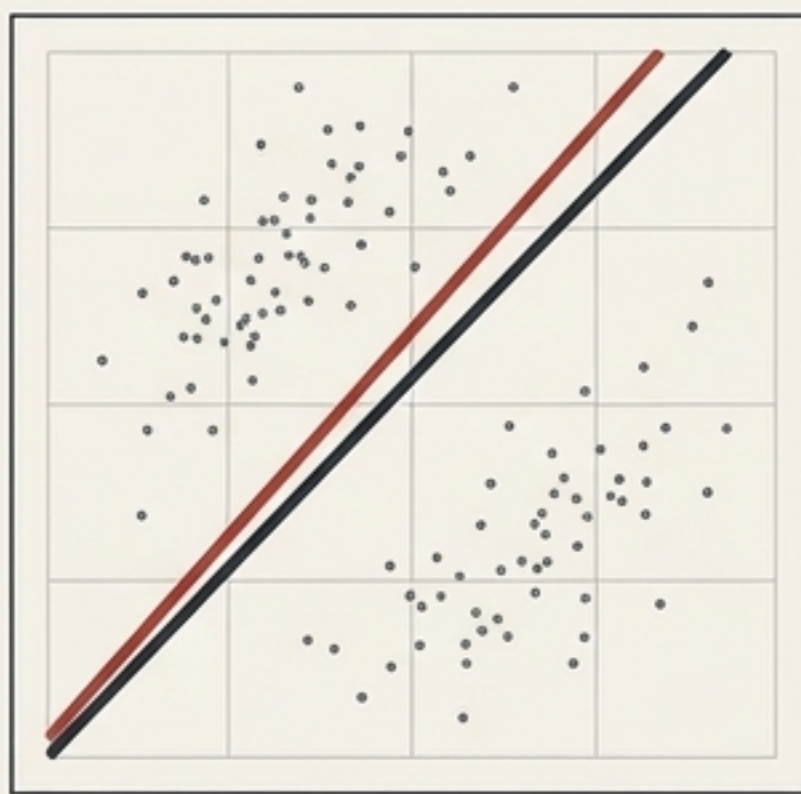
Tuning for perfection

Why isn't it an exact match instantly? Scikit-Learn uses highly optimised internal solvers.

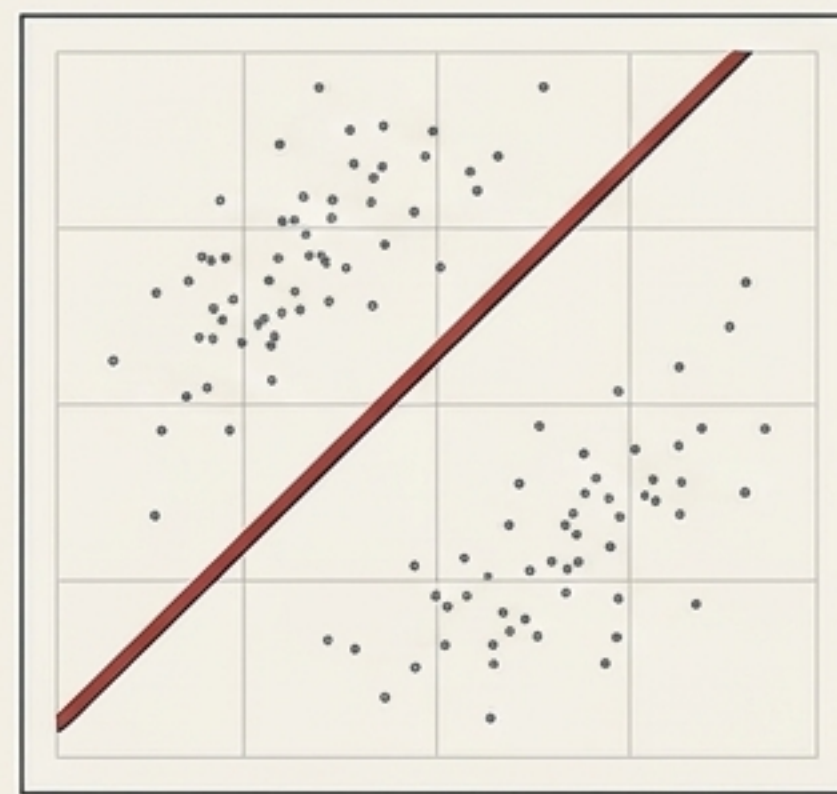
The Fix: By pushing our model to 5,000 epochs, our custom code converges directly on top of the Scikit-Learn implementation.



10 Epochs

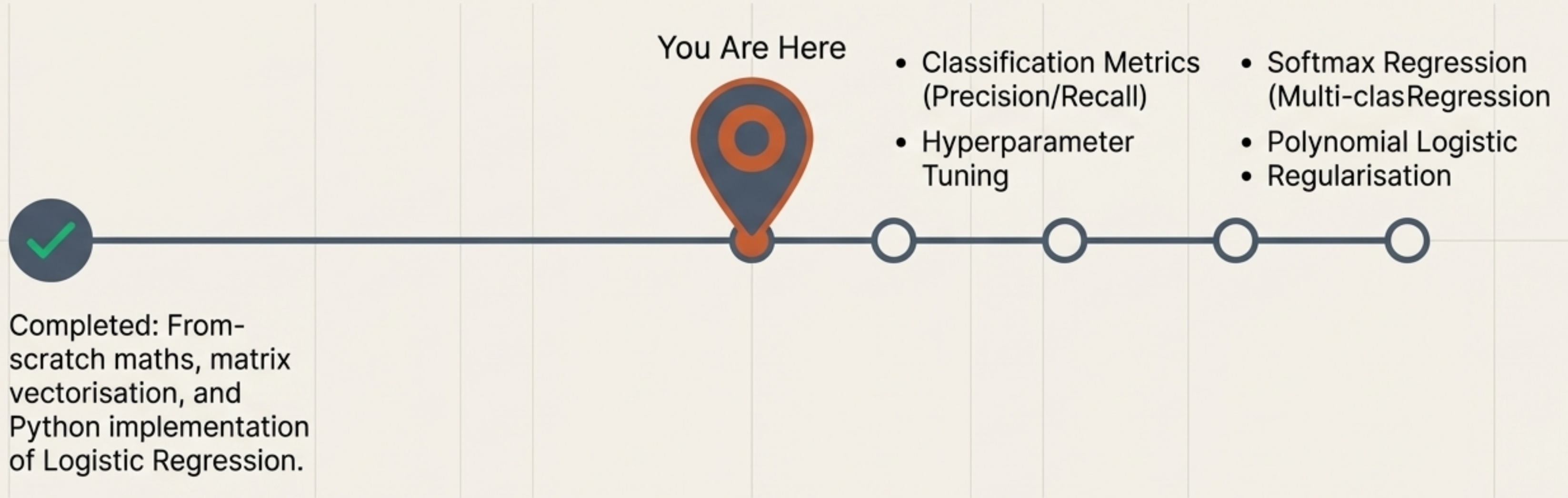


500 Epochs



5000 Epochs

What is next on the roadmap?



Understanding granular calculus makes utilising advanced machine learning libraries intuitive rather than magical.